

WhatsApp CRM App Summary

Repo-based one-page overview of the application in ``D:\CRM\whatsapp-crm``.

What It Is

A B2B SaaS web app that turns a shared WhatsApp number into a sales and support CRM. Repo docs and code show a Next.js frontend, an Express API, PostgreSQL via Prisma, and WhatsApp webhook plus real-time event handling.

Who It's For

Primary persona: sales or support teams that need shared ownership, pipeline visibility, and faster follow-up for WhatsApp conversations.

What It Does

- Shared team inbox for WhatsApp conversations, with inbox, contacts, dashboard, pipeline, follow-ups, templates, team, settings, and webhooks UI routes.
- Workspace and user authentication flows, including register, login, token refresh, current-user, workspace membership, and invite endpoints.
- Contact and conversation management backed by PostgreSQL models for contacts, conversations, messages, assignments, stages, and unread counts.
- Follow-up scheduling plus template management for repeatable replies and lead progression.
- Real-time updates over Server-Sent Events so workspace activity can stream into the frontend.
- WhatsApp webhook ingestion that creates or updates contacts, conversations, and inbound messages, then can trigger autoresponders.
- Additional API surfaces for analytics, export, broadcast, media, activity, search, and webhook logs.

How It Works

- Frontend: Next.js app with an ``AppShell`` layout (``Sidebar`` + ``Topbar``) and authenticated workspace pages under ``frontend/src/app/(app)``.
- Backend: Express server mounts public health, webhook, and auth routes, then JWT-protected ``/api/v1/*`` business routes plus protected ``/realtime/events`` SSE.
- Data: Prisma models persist users, workspaces, WhatsApp accounts, contacts, conversations, messages, follow-ups, templates, autoresponders, and webhook logs in PostgreSQL.
- Flow: Meta webhook -> ``handleWhatsAppWebhook`` -> Prisma upserts/message create -> publish workspace event -> frontend subscribes via ``EventSource`` using ``NEXT_PUBLIC_API_URL``.
- Infra in repo: Docker Compose defines PostgreSQL, optional Redis, backend, frontend, and optional Nginx reverse proxy.

How to Run

- *Backend:* ``cd backend`, `npm install`, copy `.env.example` to `.env`, set `DATABASE_URL`, `WA_VERIFY_TOKEN`, and `JWT_SECRET`, then run `npx prisma migrate dev`, `npx prisma generate`, `npx prisma db seed`, `npm run dev`.`
- *Frontend:* ``cd frontend`, `npm install`, copy `.env.example` to `.env.local`, set `NEXT_PUBLIC_API_URL=http://localhost:3001`, then run `npm run dev`.`
- *Endpoints from repo docs:* frontend on ``http://localhost:3000``, backend on ``http://localhost:3001``. Meta credentials are required for live WhatsApp integration.

Not found in repo: a complete production-ready local ``.env`` sample with all optional variables populated.